

Homework 7: Individual Assignment Database Project

Wassawin Swangjaeng 6810545905

Option C: Simple Database Application - The University Enrollment System

1. Project Scope

Purpose of the system and intended users

The system manages core university enrollment data, tracking which students belong to which departments, which courses those departments offer, and which courses students enroll in each semester, along with their scores and pass/fail status. Intended users are academic staff or department administrators who need to register students, manage courses, and track academic performance.

Information the database must manage

- Departments (dID, dName, building)
- Students (sID, sName, YEAR, dID)
- Courses (cID, cName, credit, dID)
- Enrollments (sID, cID, sem, score, pass_or_not, enroll_date)

Important activities represented

- Assigning students to departments
- Departments offering courses
- Students enrolling in courses each semester
- Recording and evaluating scores to determine pass/fail
- Generating reports such as student transcripts and average course scores

Part selected from HW#6

This project continues from the HW#6 University Enrollment System, but narrows the scope from 8 tables (which included Instructor, UndergradStudent, GraduateStudent, Scholarship, and Prerequisite) down to just 4 core tables: Dept, Student, Course, and Enroll.

2. Requirements and Business Rules

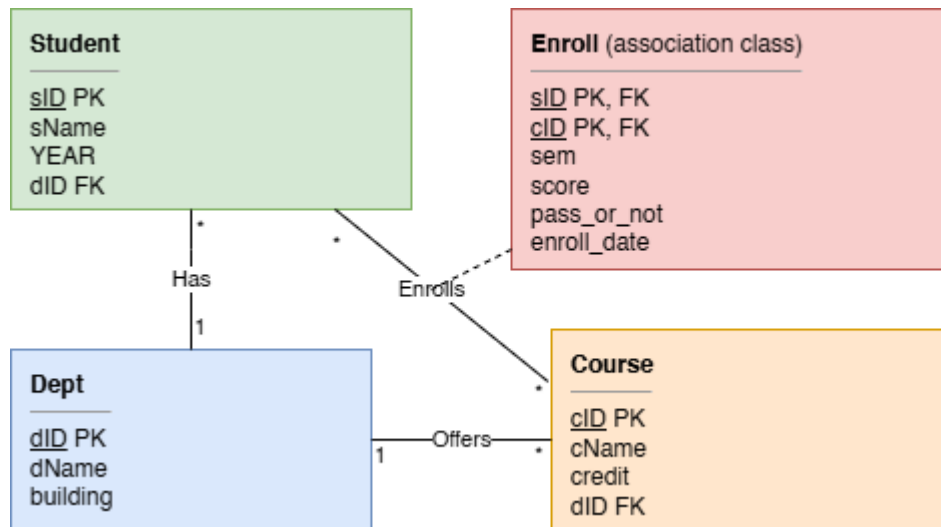
System Requirements

The university needs a database to manage academic departments, students, the courses each department offers, and student enrollment in those courses each semester. The system must track which department a student belongs to, which department owns each course, and record each student's enrollment details - including semester, score, and pass/fail outcome - for a given course.

Business Rules

- A department can have many students, but each student belongs to exactly one department.
- A department can offer many courses, but each course belongs to exactly one department.
- A student can enroll in many courses, and a course can have many students enrolled in it.
- For each enrollment, the semester, score, and pass/fail status are recorded specifically for that student-course pair.
- A student cannot enroll in the same course more than once in the system (each student-course pair is unique).
- A course must belong to an existing department; a course cannot exist without a valid department.
- A student's enrollment score, if given, must be between 0 and 100.

3. Top Down Database Design



- The Dept-Student relationship (1:M) shows that each department has many students, while each student belongs to one department (**Rule 1**).
- The Dept-Course relationship (1:M) shows that each department offers many courses, while each course belongs to one department (**Rules 2, 6**).
- The Student-Course relationship (M:N), through Enroll, shows that students can take many courses and courses can have many students (**Rule 3**).
- Enroll stores sem, score, pass_or_not, and enroll_date because they depend on a specific student-course pair (**Rule 4**).
- Enroll uses a composite primary key (sID, cID) to prevent duplicate enrollment in the same course (**Rule 5**).
- The score range (0-100) is enforced by a SQL CHECK constraint, not shown in the UML diagram (**Rule 7**).

4. Functional Dependencies and Keys

Functional Dependencies

- $dID \rightarrow dName, building$
- $cID \rightarrow cName, credit, dID$
- $sID \rightarrow sName, YEAR, dID$
- $(sID, cID) \rightarrow sem, score, pass_or_not, enroll_date$

Relation: Dept(dID, dName, building)

- Candidate key: dID
- Primary key: dID
- Foreign keys: none

dID is a short, unique department code assigned to each department, so no two departments share it, and it functionally determines dName and building.

Relation: Course(cID, cName, credit, dID)

- Candidate key: cID
- Primary key: cID
- Foreign key: dID references Dept(dID)

cID is a unique course code, so it alone determines cName, credit, and which department offers it. cName is not a candidate key because two different courses could share a name.

Relation: Student(sID, sName, YEAR, dID)

- Candidate key: sID
- Primary key: sID
- Foreign key: dID references Dept(dID)

sID is a unique student ID assigned to each student, so it alone determines sName, YEAR, and department. sName is not a candidate key since two students could share a name.

Relation: Enroll(sID, cID, sem, score, pass_or_not, enroll_date)

- Candidate key: (sID, cID)
- Primary key: (sID, cID)
- Foreign keys: sID references Student(sID), cID references Course(cID)

Neither sID nor cID alone is unique in this table, since a student enrolls in many courses and a course has many students. The pair (sID, cID) is unique because a student can enroll in a given course only once, so together they determine sem, score, pass_or_not, and enroll_date.

5. Normalization

Bottom-up normalization from mega relational database

1. Initial Relation

StudentCourseEnrollment(sID, sName, YEAR, dID, dName, building, cID, cName, credit, sem, score, pass_or_not, enroll_date)

2. Candidate Key and Primary Key

Candidate key: (sID, cID) Selected primary key: (sID, cID)

3. Functional Dependencies

sID → sName, YEAR, dID

dID → dName, building

cID → cName, credit, dID

(sID, cID) → sem, score, pass_or_not, enroll_date

4. First Normal Form (1NF)

All attributes contain atomic (single) values, so the relation satisfies 1NF.

5. Second Normal Form (2NF)

The partial dependencies are:

sID → sName, YEAR, dID

cID → cName, credit, dID

New relations are:

Students(sID, sName, YEAR, dID, dName, building)

Courses(cID, cName, credit, dID, dName, building)

Enrollments(sID, cID, sem, score, pass_or_not, enroll_date)

6. Third Normal Form (3NF)

Transitive dependency:

sID → dID → dName, building

cID → dID → dName, building

New separate relation:

Departments(dID, dName, building)

7. Final Normalized Relations

Departments(dID, dName, building)

Students(sID, sName, YEAR, dID)

Courses(cID, cName, credit, dID)

Enrollments(sID, cID, sem, score, pass_or_not, enroll_date)

Top-down UML design

Departments(dID, dName, building)

Students(sID, sName, YEAR, dID)

Courses(cID, cName, credit, dID)

Enrollments(sID, cID, sem, score, pass_or_not, enroll_date)

Required Reflection Question

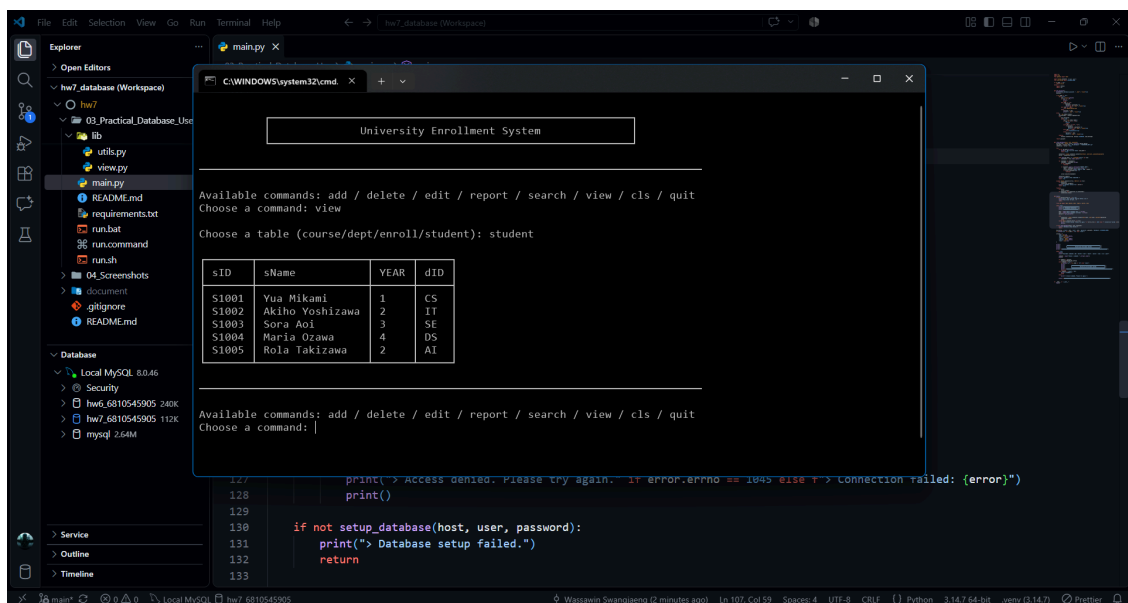
Do the top-down UML design and bottom-up normalization lead to the same or equivalent schema? Explain briefly.

Yes. Both approaches lead to equivalent schemas. The UML design defines the relationships, while normalization removes redundancy. Both result in the same four tables: Departments, Students, Courses, and Enrollments.

6. Implementation screenshots

```
1 CREATE DATABASE IF NOT EXISTS hw7_6810545905;
2
3 USE hw7_6810545905;
4
5 CREATE TABLE IF NOT EXISTS Dept (
6     dID CHAR(2) PRIMARY KEY,
7     dName VARCHAR(30) NOT NULL,
8     building CHAR(2) NOT NULL
9 );
10
11 CREATE TABLE IF NOT EXISTS Student (
12     sID CHAR(5) PRIMARY KEY,
13     sName VARCHAR(30) NOT NULL,
14     YEAR INT NOT NULL CHECK (YEAR BETWEEN 1 AND 4),
15     dID CHAR(2) NOT NULL,
16     FOREIGN KEY (dID) REFERENCES Dept (dID)
17         ON DELETE CASCADE
18         ON UPDATE CASCADE
19 );
20
21 CREATE TABLE IF NOT EXISTS Course (
22     cID CHAR(6) PRIMARY KEY,
23     cName VARCHAR(30) NOT NULL,
24     credit INT NOT NULL CHECK (credit > 0),
25     dID CHAR(2) NOT NULL,
26     FOREIGN KEY (dID) REFERENCES Dept (dID)
27         ON DELETE CASCADE
28         ON UPDATE CASCADE
29 );
30
31 CREATE TABLE IF NOT EXISTS Enroll (
32     sID CHAR(5),
33     cID CHAR(6),
34     sem CHAR(2) NOT NULL,
35     score INT NULL CHECK (score BETWEEN 0 AND 100),
36     pass_or_not CHAR(1) NULL CHECK (pass_or_not IN ("Y", "N")),
37     enroll_date DATE NOT NULL,
38     PRIMARY KEY (sID, cID),
39     FOREIGN KEY (sID) REFERENCES Student (sID)
40         ON DELETE CASCADE
41         ON UPDATE CASCADE,
42     FOREIGN KEY (cID) REFERENCES Course (cID)
43         ON DELETE CASCADE
44         ON UPDATE CASCADE
45 );
46
47 INSERT INTO Dept (dID, dName, building) VALUES
48 ("CS", "Computer Science", "K1"),
49 ("IT", "Information Tech", "K2"),
50 ("SE", "Software Engineer", "K3"),
51 ("DS", "Data Science", "K4"),
52 ("AI", "Artificial Intel", "K5");
53
54 INSERT INTO Student (sID, sName, YEAR, dID) VALUES
55 ("S1001", "Yua Mikami", 1, "CS"),
56 ("S1002", "Akiho Yoshizawa", 2, "IT"),
57 ("S1003", "Sora Aoi", 3, "SE"),
58 ("S1004", "Maria Ozawa", 4, "DS"),
59 ("S1005", "Rola Takizawa", 2, "AI");
60
61 INSERT INTO Course (cID, cName, credit, dID) VALUES
62 ("CS2001", "Database", 3, "CS"),
63 ("IT2001", "Networking", 3, "IT"),
64 ("SE2001", "Software Design", 3, "SE"),
65 ("DS2001", "Data Analysis", 3, "DS"),
66 ("AI2001", "Machine Learning", 3, "AI");
67
68 INSERT INTO Enroll
69 (sID, cID, sem, score, pass_or_not, enroll_date)
70 VALUES
71 ("S1001", "CS2001", "1", 85, "Y", "2026-01-01"),
72 ("S1001", "IT2001", "1", 78, "Y", "2026-01-02"),
73 ("S1002", "IT2001", "1", 92, "Y", "2026-01-03"),
74 ("S1002", "SE2001", "1", 68, "N", "2026-01-04"),
75 ("S1003", "SE2001", "1", 88, "Y", "2026-01-05"),
76 ("S1003", "DS2001", "1", 75, "Y", "2026-01-06"),
77 ("S1004", "DS2001", "1", 95, "Y", "2026-01-07"),
78 ("S1004", "AI2001", "1", 82, "Y", "2026-01-08"),
79 ("S1005", "AI2001", "1", 90, "Y", "2026-01-09"),
80 ("S1005", "CS2001", "1", 73, "Y", "2026-01-10");
81
```

7. Application screenshots



8. Validation and Testing

Insert (Add new department)

Available commands: add / delete / edit / report / search / view / cls / quit
Choose a command: add

Choose a table (course/dept/enroll/student): dept
Enter dID: AL
Enter dName: Algorithm
Enter building: K6
> Record added successfully.

Available commands: add / delete / edit / report / search / view / cls / quit
Choose a command: view

Choose a table (course/dept/enroll/student): dept

dID	dName	building
AI	Artificial Intel	K5
AL	Algorithm	K6
CS	Computer Science	K1
DS	Data Science	K4
IT	Information Tech	K2
SE	Software Engineer	K3

Available commands: add / delete / edit / report / search / view / cls / quit
Choose a command: |

Update (Edit department data)

Available commands: add / delete / edit / report / search / view / cls / quit
Choose a command: edit

Choose a table (course/dept/enroll/student): dept
Enter dID: AL
Available fields: dName, building
Choose a field to update: building
Current value: K6 | Enter new value: K7
> Record updated successfully.

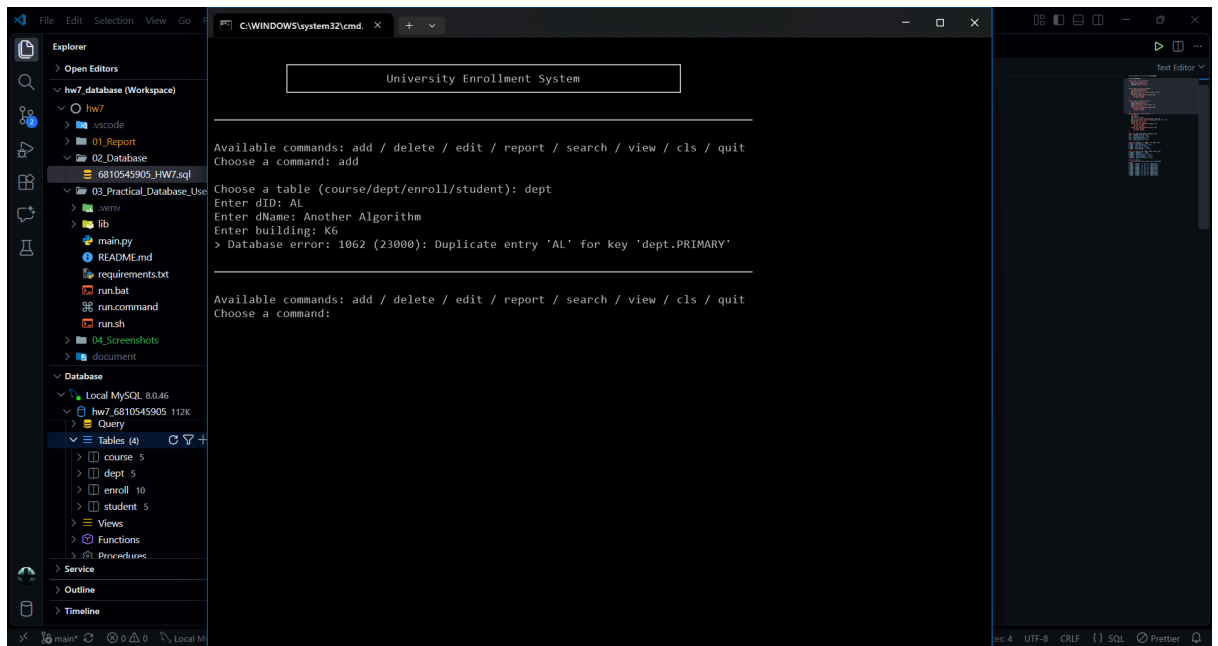
Available commands: add / delete / edit / report / search / view / cls / quit
Choose a command: view

Choose a table (course/dept/enroll/student): dept

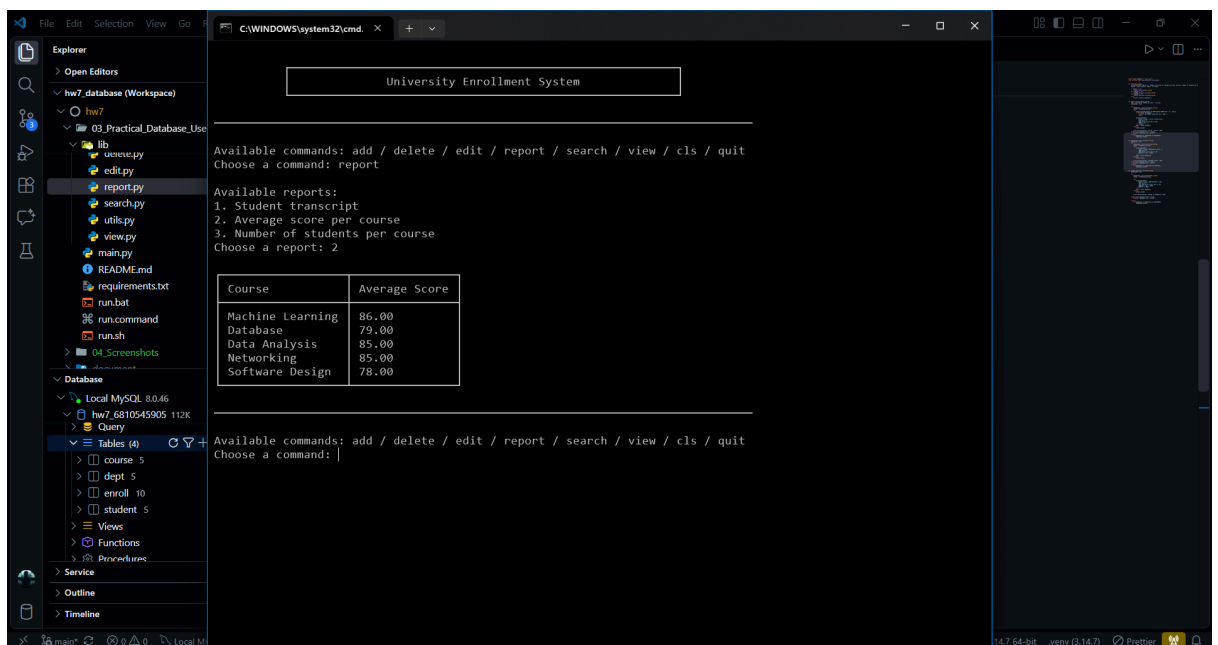
dID	dName	building
AI	Artificial Intel	K5
AL	Algorithm	K7
CS	Computer Science	K1
DS	Data Science	K4
IT	Information Tech	K2
SE	Software Engineer	K3

Available commands: add / delete / edit / report / search / view / cls / quit
Choose a command: |

Constraint (duplicate primary key)



Join (Enroll and Course)



The database was tested by inserting or updating records and checking whether the changes were saved correctly. Database constraints were tested using invalid data to confirm that the system rejected it. The query results were also checked to ensure that data from multiple tables was displayed correctly.